

TokTickIT

Lab 1 full-stack vertical slice

Stack	React, TypeScript, Vite, Bootstrap, Express, Prisma, PostgreSQL
Workflow	Bun workspaces, feature branches, GitHub Issues, Pull Requests
Evidence	Tests, API contracts, database seed, and human-checked App Demo

Contents

1	Answer Part 1: Git Use with Engineering Workflow	2
2	Answer Part 2: Tests	6
3	Answer Part 3: AI Use and Reflection	7
4	Answer Part 4: App Demo	8

1 Answer Part 1: Git Use with Engineering Workflow

The repository uses four parent Lab 1 issues and native sub-issues. Work is performed on feature branches and targets `lab1-staging`; the stable branch is `main`.

Parent issue	Feature branch	Pull Request
#13 foundation	<code>feature/1-project-foundation</code>	PR #44
#14 health	<code>feature/2-health-check</code>	PR #46
#15 categories	<code>feature/3-category-seed</code>	PR #45
#16 category list	<code>feature/4-category-list</code>	PR #47

Repository	<code>ovenmakemeheat/toktickit</code>
Project board	TokTickIT Lab 1
Integration branch	<code>lab1-staging</code>
Stable branch	<code>main</code>
Integration status	All four feature PRs are merged into <code>lab1-staging</code> ; no staging-to-main PR exists in the audited GitHub state.

Student and peer reviewer

Student GUNTEE DOUNGMANEE ID: 67070501003 GitHub: <code>ovenmakemeheat</code>	Peer reviewer POLWARIT WATTTHANAHEMMARAT ID: 67070501067 GitHub: <code>MadMax168</code>
--	--

- Repository: <https://github.com/ovenmakemeheat/toktickit>
- Project board: <https://github.com/users/ovenmakemeheat/projects/1/views/2>
- The parent issue checklists are the acceptance source; native sub-issues carry implementation, testing, and evidence slices.
- Positive peer-review records for PRs #44-#47 are audited with the author responses.
- Full review details are in `docs/lab-01/reviewer.md`; PRs #44-#47 are merged into `lab1-staging`.
- Unresolved automated threads on PRs #44, #45, and #47 are recorded separately and are not peer-review rejections.
- Reciprocal review target: `MadMax168/TikTockIT`; its public pull-request list was empty at audit time, so reciprocal review evidence remains a human action.

Student-peer reviewer conversation

Pull request	Peer reviewer message	Student response
#44 foundation (Merged)	"All things good for me"	Acknowledged the lockfile comment and stated that the synchronized lockfile would be regenerated and committed.
#45 category seed (Merged)	"Good for me"	"LGTM"
#46 health check (Merged)	"Nothing to change, Good for Me"	"Thanks for the review!"
#47 category list (Merged)	"All Be Good for Me"	"Thanks!" The pull request was merged into <code>lab1-staging</code> .

GitHub records these peer submissions as *COMMENTED*; they are conversation evidence, not formal *APPROVE* events.

Repository workflow screenshots

The following captures provide human-checked evidence for the board state, merge history, repository structure, and automated test output. GitHub captures were taken with browser-use CLI from the relevant `lab1-staging` and `main` views.

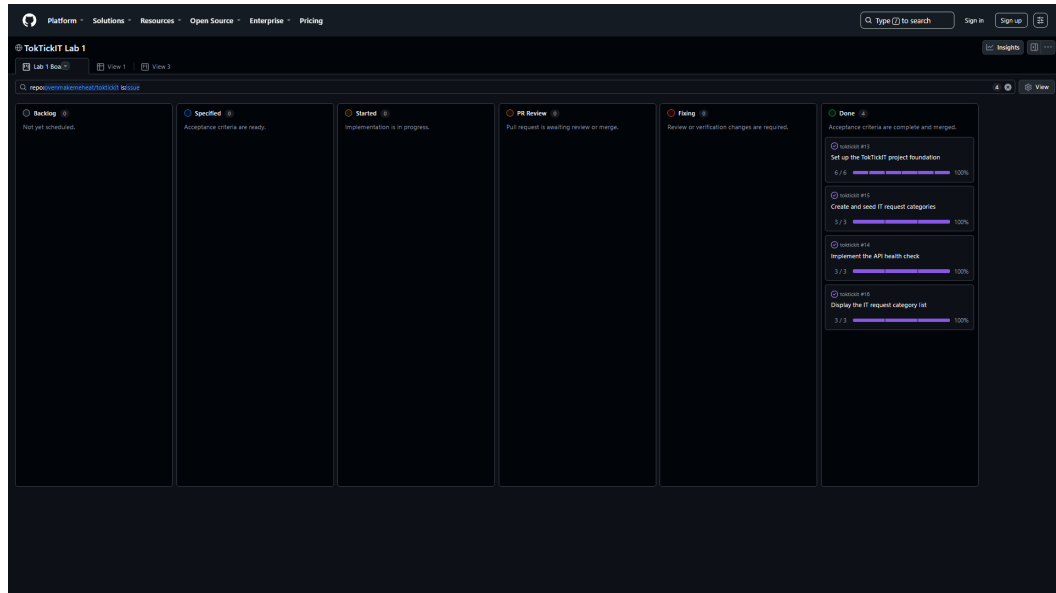


Figure 1: Lab 1 project board: six workflow columns, all four parent issues in Done, and 100% sub-issue progress.

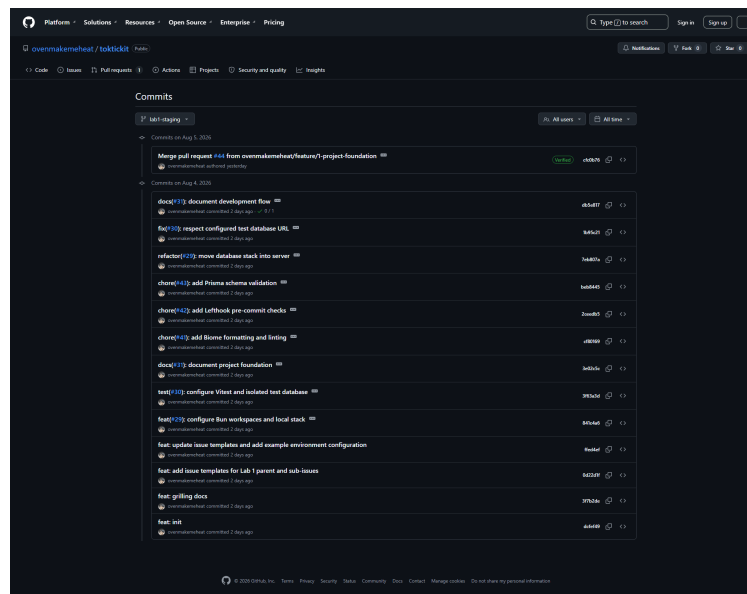


Figure 2: Commit and merge history on lab1-staging, including the PR #44 merge and issue-referenced commits.

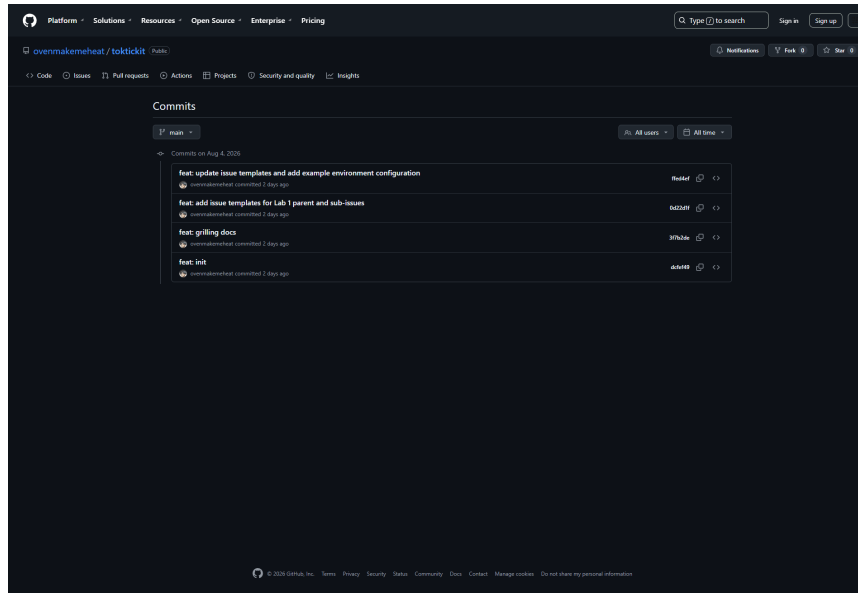


Figure 3: Current main history: it still contains only the initial setup commits, so staging-to-main integration remains a human action.

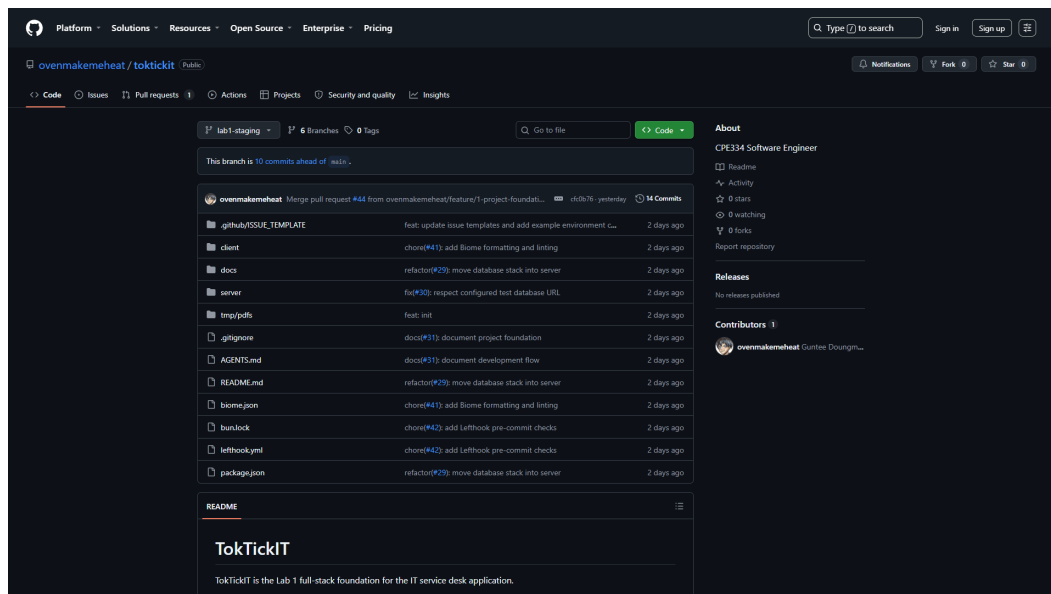


Figure 4: Repository tree showing the client, server, documentation, agent instructions, and root tooling.

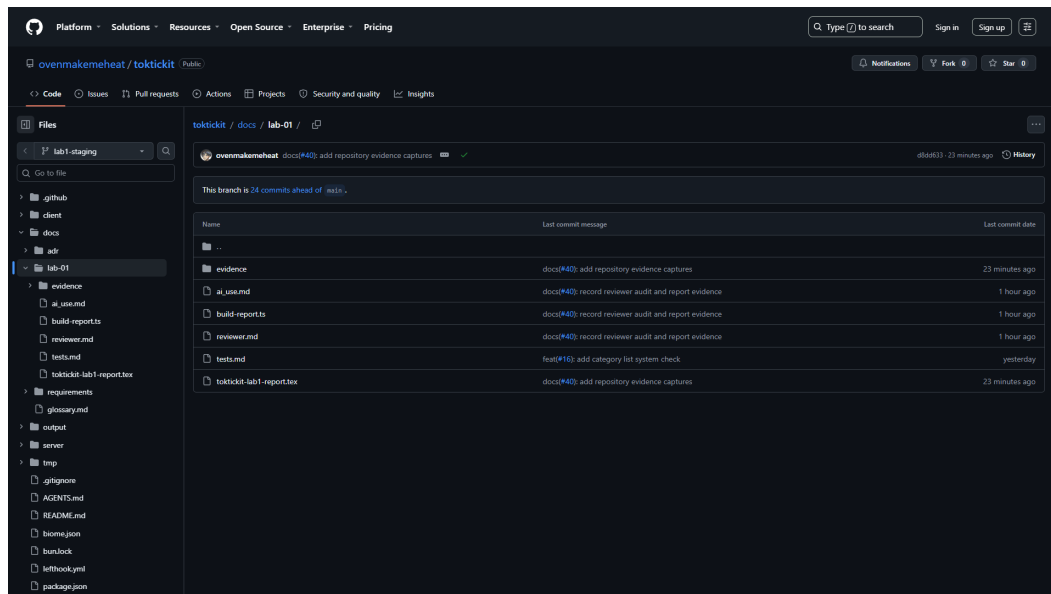


Figure 5: Nested documentation tree showing `tests.md`, `reviewer.md`, `ai_use.md`, and the LaTeX report source.

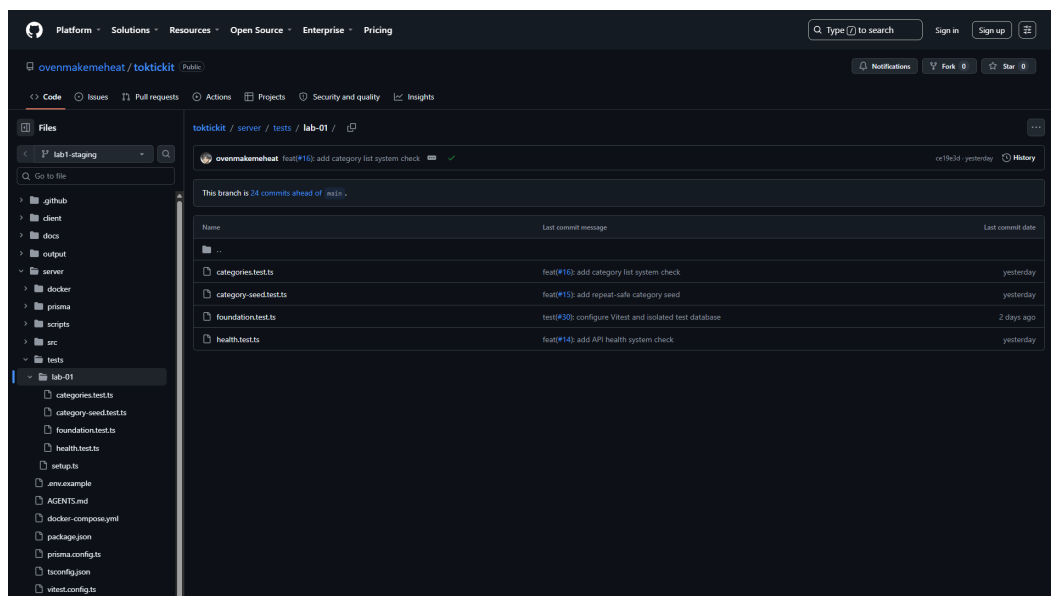


Figure 6: Nested server test tree showing the Lab 1 API, seed, foundation, and setup test files.

```

PS C:\Users\Gunte\Workspace\Workspace\works\CPE\toktickit> bun run test
$ bun run test:client && bun run test:server
$ bun run --cwd client test
$ vitest run

RUN v3.2.7 C:\Users\Gunte\Workspace\Workspace\works\CPE\toktickit\client

✓ src/App.test.tsx (3 tests) 341ms
✓ TokTickIT system-check screen > renders the application heading and Check System action 142ms
✓ TokTickIT system-check screen > transitions from checking to Online and renders returned categories 92ms
✓ TokTickIT system-check screen > clears stale categories and shows Offline when either API request fails 105ms

Test Files 1 passed (1)
Tests 3 passed (3)
Start at 23:12:41
Duration 3.97s (transform 123ms, setup 569ms, collect 347ms, tests 341ms, environment 1.54s, prepare 506ms)

$ bun run --cwd server test
$ vitest run

RUN v4.1.10 C:\Users\Gunte\Workspace\Workspace\works\CPE\toktickit\server

stdout | tests/lab-01/categories.test.ts
◊ injected env (7) from .env // tip: ⚠enable debugging { debug: true }

stdout | tests/lab-01/category-seed.test.ts
◊ injected env (7) from .env // tip: ⚠encrypted .env [www.dotenvx.com]

stdout | tests/lab-01/health.test.ts
◊ injected env (7) from .env // tip: ⚠custom filepath { path: '/custom/path/.env' }

stdout | tests/lab-01/foundation.test.ts
◊ injected env (7) from .env // tip: ~ auth for agents [www.vestauth.com]

✓ tests/lab-01/category-seed.test.ts (2 tests) 161ms
✓ tests/lab-01/health.test.ts (1 test) 35ms
✓ tests/lab-01/categories.test.ts (1 test) 115ms
✓ tests/lab-01/foundation.test.ts (2 tests) 37ms

Test Files 4 passed (4)
Tests 6 passed (6)
Start at 23:12:47
Duration 1.53s (transform 194ms, setup 228ms, import 2.20s, tests 347ms, environment 1ms)

```

Figure 7: Terminal evidence from the integrated branch: the client suite passes 3 tests and the server suite passes 6 tests.

Rendered README.md

Purpose	TokTickIT is the Lab 1 full-stack foundation for the IT service desk application.
Stack	Bun workspaces; React, TypeScript, Vite, Bootstrap; Node-compatible Express and TypeScript; PostgreSQL 16 through Docker Compose; Prisma; Biome; Lefthook; Vitest; and Supertest.
Prerequisites	Bun 1.3 or newer and Docker with Compose.
Setup	<code>bun install</code> ; copy <code>server/.env.example</code> to <code>server/.env</code> ; install hooks; start the database; generate and validate Prisma.
Database	The stack is owned by <code>server/</code> ; Compose is under <code>server/docker-compose.yml</code> , PostgreSQL initialization is under <code>server/docker/</code> , and Prisma files are under <code>server/prisma/</code> .
Seed and tests	<code>bun run db:migrate</code> , <code>bun run db:seed</code> , <code>bun run db:test:setup</code> , then <code>bun run verify</code> .
Run	<code>bun run dev</code> ; client at http://localhost:5173 , API at http://localhost:3000 , and PostgreSQL at <code>localhost:5432</code> .
Scope	Lab 1 proves the client, API, Prisma, PostgreSQL, and test tooling as one vertical slice. Authentication, tickets, uploads, and later-lab screens are out of scope.

Rendered .gitignore

Dependencies	<code>node_modules/</code> , <code>.bun/</code> , and package-manager stores.
Environment	<code>.env</code> and <code>.env.*</code> , except <code>.env.example</code> .
Build and cache	<code>dist/</code> , <code>build/</code> , <code>.vite/</code> , coverage, TypeScript build info, and logs.
Local state	<code>*.db</code> , <code>*.db-journal</code> , <code>tmp/</code> , and OS metadata.
Editor and agent files	Local VS Code settings, <code>.claude/</code> , and <code>CLAUDE.md</code> .
LaTeX artifacts	<code>*.aux</code> , <code>*.fdb_latexmk</code> , <code>*.fls</code> , <code>*.synctex.gz</code> , <code>*.toc</code> , <code>*.out</code> , and <code>*.xdv</code> .

2 Answer Part 2: Tests

The test matrix is kept in `docs/lab-01/tests.md`. The application boundary is tested through the exported Express app and user-observable React behavior.

ID	Test file	Tool	Proof	Status
API-01	<code>server/tests/lab-01/health.test.ts</code>	Supertest	Health returns HTTP 200 and the exact TokTickIT API payload.	Passing
API-02	<code>server/tests/lab-01/categories.test.ts</code>	Supertest	Categories return seeded ID/name pairs in ascending ID order.	Passing
UI-01	<code>client/src/App.test.tsx</code>	Vitest	TokTickIT heading and Check System action render.	Passing
UI-02	<code>client/src/App.test.tsx</code>	Vitest	Checking transitions to Online and renders returned categories.	Passing
UI-03	<code>client/src/App.test.tsx</code>	Vitest	API failure clears stale categories and shows the safe error.	Passing

The additional foundation smoke test is at `server/tests/lab-01/foundation.test.ts`; it verifies the exported Express app and typed environment configuration. This table is the rendered test traceability from `docs/lab-01/tests.md`.

Verification command: `bun run verify`. It covers Biome, Lefthook validation, Prisma validation, type checks, both workspace test suites, and both workspace builds.

3 Answer Part 3: AI Use and Reflection

Tool and model

Tool	OpenAI Codex coding agent in ChatGPT
Model	GPT-5
Supporting tools	GitHub connector, browser-use CLI, Bun, XeLaTeX, and Poppler

Selected prompts and effects

ID	Selected prompt	Effect
P1	<i>“read @docs/requirements, \$grill-with-docs”</i>	Loaded the requirements and used the grilling workflow to identify scope and risks.
P2	<i>“now follow the plan, first create the issues”</i>	Converted the requirements into the Lab 1 issue structure.
P3	<i>“all subissues should be inside the main 4 issues”</i>	Kept exactly four parent issues and attached implementation work as native sub-issues.
P4	<i>“please keep insturction in AGENTS.md strictly about don’t close any PR or main issue you can only do check the checklists or closing subissue, closing main issue / PR close is human job”</i>	Established human-only rules for pull-request and main-issue state changes.
P5	<i>“add biome, left-hook, prisma validation on git sub-issues of issue 1”</i>	Added the requested quality gates and connected them to the foundation work.
P6	<i>“yes, and pdf should be built in latex”</i>	Chose a reproducible XeLaTeX report source and PDF build command.
P7	<i>“do capture those for me from http://localhost:5173/”</i>	Defined the required loading, success, and failure evidence for the running app.
P8	<i>“do use browseruse cli”</i>	Used browser-use CLI to verify the visible UI states and save the captures.
P9	<i>“this is student - reviewer details Student ... also checkout all pr and review records; update reviewer part”</i>	Audited PRs #44-#47 and recorded student, peer, review comments, responses, and unresolved automated threads.

Reflection

The prompts became more useful as they added concrete constraints: Bun, the server-owned database stack, four parent issues, native sub-issues, acceptance criteria, and human-only pull-request state changes. Those constraints kept the implementation aligned with Lab 1 instead of expanding into later features.

The AI-assisted workflow still required human-in-the-loop checks. The browser capture was inspected visually; an initial loading attempt was detected as a completed success state and recaptured correctly. The final PDF was rendered page by page and checked for readable evidence, clipping, overlap, and excessive empty space. GitHub

review records were also checked directly so positive peer comments were not mislabeled as formal approvals, and unresolved automated threads were preserved rather than reported as fixed.

4 Answer Part 4: App Demo

The system check makes two relative API requests through the Vite proxy: `/api/health` for health and `/api/categories` for categories. The success state proves that health is online and that the four categories came from PostgreSQL. The failure state proves that the user receives a safe error and stale categories are cleared.

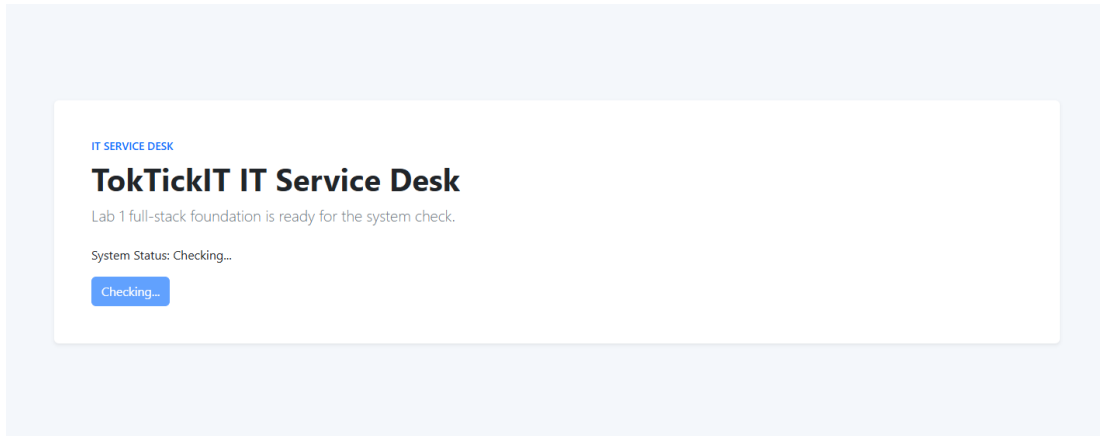


Figure 8: Loading state: the button and status show that the system check is in progress.

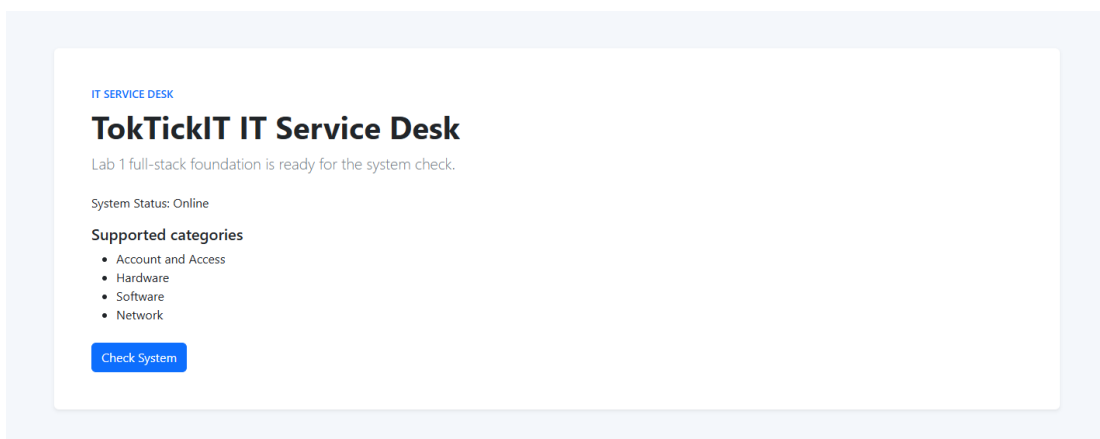


Figure 9: Success state: Online and the database-backed categories are visible in order.

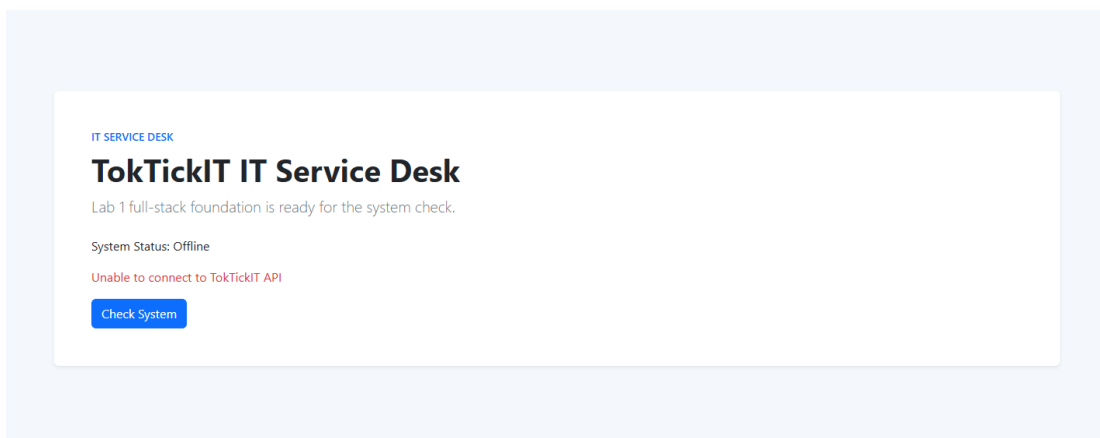


Figure 10: Failure state: the API is unavailable and the user-safe error is shown.